

遮住光，電腦怎麼知道？

Week 3：用電表看到電壓，再讓 ESP32 讀到變化。

用一張不透光的紙遮住光敏模組，觀察電表與電腦讀值如何改變，再讓程式印出「室內光 / 遮光」。



我們依序做	要看到的結果
用電表量 3V3 和 GPIO5	電源電壓，以及程式切換的高、低電壓
接光敏模組，再用紙遮住	電表讀值及電腦數字是否改變
用自己的數據設定規則	畫面出現「室內光 / 遮光 / 未判定」
交換兩顆固定電阻	解釋光敏模組為何能改變電壓

這個作品使用電表、光敏模組與按鈕，在電腦保存光線資料。

量測欄記下自己的數字與條件；尚未量測的欄位留白，不抄示例當實測。

開啟本週的四支程式

1. 使用隨本講義提供的完整 IoT 資料夾，內含本週四支完整程式。
2. 若取得 ZIP 檔，在檔案總管右鍵 ZIP，選「全部解壓縮」。沒有課程檔案時，先取得與本講義相符的檔案包。
3. 打開解壓縮後的資料夾，再開 **IOT_Introduction**。不要直接在 ZIP 預覽視窗操作。
4. 依下表找到當段程式；在 Arduino IDE 選「檔案 → 開啟」，選該資料夾中的同名 **.ino**。瀏覽器裡看到程式文字，不等於已在 IDE 開啟。
5. 修改前選「檔案 → 另存新檔」，存進自己的練習資料夾。按 **Ctrl+F** 搜尋講義指定的設定名稱，改等號右邊的值，保留分號，再按 **Ctrl+S**。

本週依序開這些檔案

以下從 IOT_Introduction 資料夾開始找；四支程式也完整列在各自操作段落。

- `examples/week03_gpio_voltage_cycle/week03_gpio_voltage_cycle.ino`
- `examples/week03_ky018_raw/week03_ky018_raw.ino`
- `examples/week03_light_classifier/week03_light_classifier.ino`
- `docs/teaching_drafts/week3_redesign/button_light_capture/button_light_capture.ino`

換另一支程式時，原來修改的設定**不會自動帶過去**。依該段設定表逐項填寫；不把短片段當成完整程式。

若下載內容和講義列出的檔名不一致，先取得相符的課程資料夾，不猜替代檔。開啟檔案時，ESP32 的 USB 與外部電源保持拔除。

清空接線，準備量電壓

GPIO4、GPIO5 這週要換用途，不能沿用計數器接線。

1. 拔掉 ESP32 的 USB，移除電池及其他外部供電，確認 PWR 燈熄滅。
2. 電表轉 OFF，移開表筆。拍下舊作品接法後，拆除開發板所有外接線。
3. 拿空的 400 孔麵包板。板子放在乾燥、不導電桌面，不直接插進麵包板。
4. 按鈕先收起來，完成電表、感測器與分壓實作後，再接回按鈕取樣。

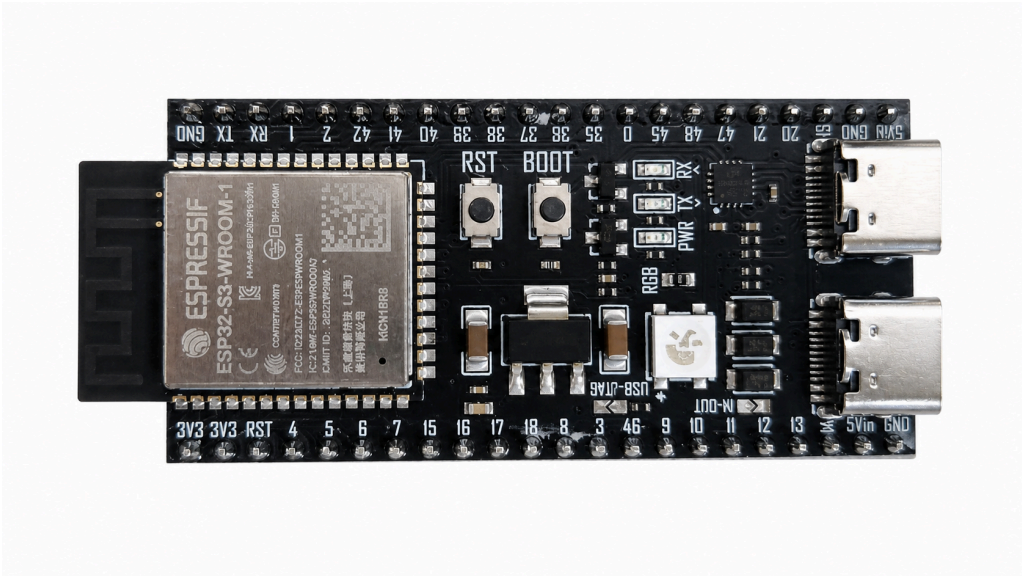
現在拿出來	數量
圖示 ESP32-S3 開發板、USB 資料線、筆電	各 1
萬用電表、400 孔麵包板	各 1
公對母 / 公對公 / 母對母杜邦線	各數條
KY-018 光敏模組、不透光的紙	各 1
電阻包中的 1 k Ω 、10 k Ω	各 1 顆
Week2 使用過的四腳按鈕	1 顆

以後每次改線：先移開表筆 → 拔 USB 與其他電源 → 確認 PWR 熄滅 → 電表 OFF → 才移線。關閉 Serial Monitor 不等於斷電。

本週三個用途：3V3 給小型感測模組供電；GPIO5 暫時輸出高 / 低電壓；GPIO4 稍後讀光敏訊號。這些用途不是同時接上舊按鈕。

只找四個腳位

天線在左，USB 在右。看印字，不是數第幾根針。



課堂實物正面：YD-ESP32-S3 Type-A V1.5 · N16R8。

找哪個印字	這張照片的位置	用途
GND	上排最左端，靠天線	共同參考點
3V3	下排最左端	約 3.3 V 電源
5	下排，在 4 與 6 之間	稍後輸出電壓
4	下排，在 RST 與 5 之間	稍後讀光敏訊號

不要接 5Vin。 這片板的 IN-OUT 焊盤未橋接時，5Vin 不是 USB 的 5 V 輸出。本週不焊接、不橋接它。

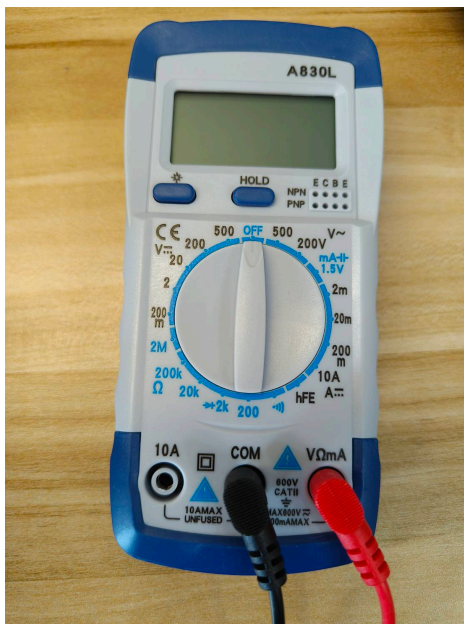
本稿接法對應圖示板型。不同板型、看不清印字或課程尚未確認 GPIO 用法時，保持斷電，一起核對。照片不能證明所有同名商品相同。

本段量 3V3 對 GND 的電壓，GPIO4、GPIO5 保持未接線。

黑筆、紅筆、旋鈕各放哪裡？

這次量電壓，不是量電流，也不是聽通斷聲。

V 是電壓單位「伏特」。20 V 檔是量測範圍，不是電表會輸出 20 V。



課堂 A830L 實物照片。照片旋鈕在 OFF；量測前仍須自己選檔。

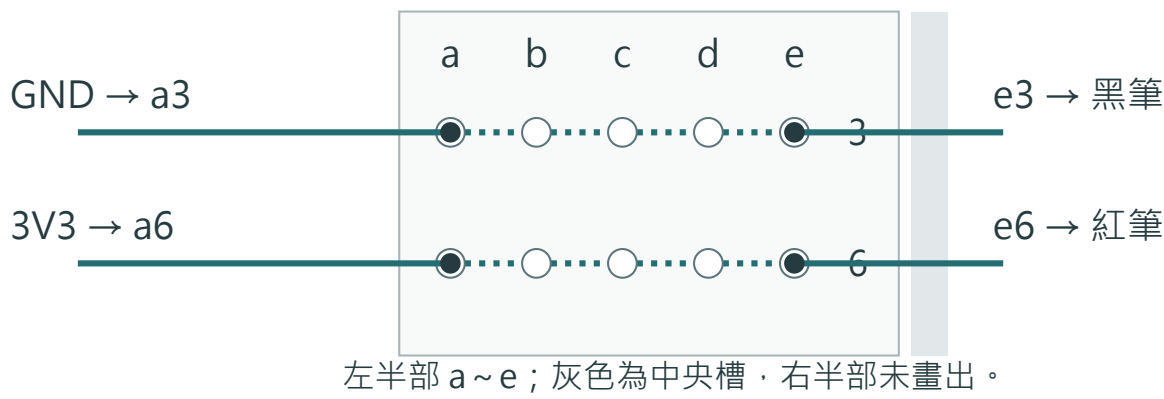
位置	這次設定
黑表筆插孔	COM；此表下方中間
紅表筆插孔	VΩmA；此表下方右邊
旋鈕	直流電壓 20 V：找 V 與實線 / 虛線符號，不是 V~
10A 插孔	保持空著

COM 是電表的共用參考端，並不會自己連到板子的 GND。等一下要由黑表筆碰到 GND 的延長線，兩者才連起來。

本週完全不使用 A / mA 電流檔，也不量插座或市電。表型不同時看自己的符號與說明，不照照片角度猜。先設好，再將表筆放旁邊。

兩條線，把電源位置引出來

麵包板只讓表筆比較好碰，不會自己製造電源。



先後	起點 → 終點	接頭
1	ESP32 GND → a3	母頭套排針，公頭插孔
2	ESP32 3V3 → a6	母頭套排針，公頭插孔
3	e3 → 一條公對公線的自由端	黑筆稍後碰這個自由端
4	e6 → 另一條公對公線的自由端	紅筆稍後碰這個自由端

同列 **a~e 相通**。a3 與 e3 是同一位置的延伸；a6 與 e6 是另一組。第 3 列和第 6 列不可用線直接相連。兩個自由金屬端要分開固定。

一起核對表格及列號，確認沒有其他線或元件跨接第 3、6 列。若做通斷複查，只能在拔 USB 時做；供電與 GND 持續蜂鳴時保持斷電排查。**不蜂鳴也不是完整安全保證。**

表筆不要硬塞進麵包板。不要讓延長線金屬端互碰，也不要一邊通電一邊整理線。

黑筆固定，紅筆只碰一個端點

先猜：3V3 相對 GND，會接近 0 V 還是 3.3 V？

1. 電表確認為直流 20 V。插上板背標示 COM 接頭的 USB。
2. 黑筆碰 e3 延長線的自由金屬端。
3. 紅筆碰 e6 延長線的自由金屬端，不碰旁邊排針。
4. 接觸穩定後記錄完整畫面與單位，再移開紅筆。



顯示例：約 +3.3 V；兩端不可用普通線直接互接。

黑筆	紅筆	檔位	預期	我的結果
e3 · GND	e6 · 3V3	直流 20 V	約 +3.3 V	___ V

預期為接近 +3.3 V 的正值，不要求每片板或每支電表完全相同；記錄自己看到的數字。

如果是負數：先移開表筆，核對紅黑測點，不交換板子的供電線。**如果接近 0 V：**移開表筆、斷電，核對 USB、PWR、3V3 印字與列號。

電壓是比較兩個位置

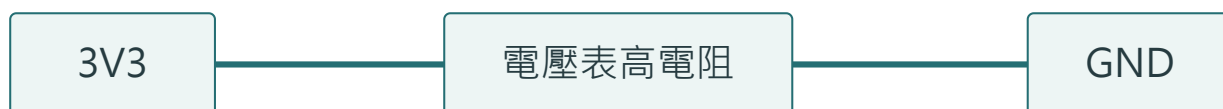
不是「紅筆裡面有 3.3 V」，也不是「流過了 3.3 A」。

電表在直流電壓檔顯示：**紅筆位置的電位** – **黑筆位置的電位**。本週把 GND 當共同的 0 V 參考。

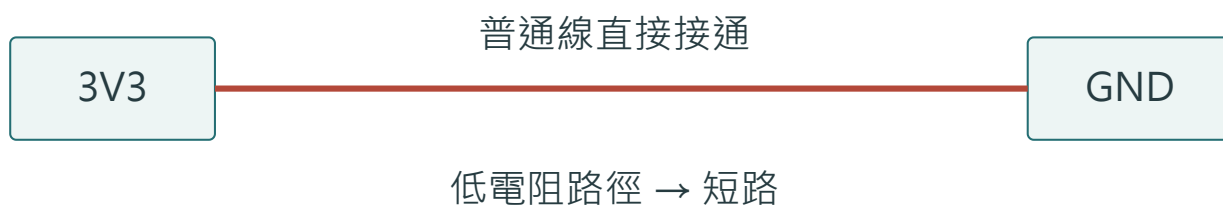
紙上情境，不必重新接線	計算	顯示
紅筆 3V3、黑筆 GND	$3.3 - 0$	+3.3 V
紅筆 GND、黑筆 3V3	$0 - 3.3$	-3.3 V
兩筆碰同一個 3V3 節點	$3.3 - 3.3$	0 V

第三列不表示板子沒電，只表示兩個測點的電位相同。GND 也不是讓電流憑空消失的地方，它接回電源回路。

正常量測：



不要實作：



為什麼電表跨接 3V3 和 GND 不等於短路？電壓檔內部有很大的輸入電阻，只取很小的電流。普通杜邦線的電阻很低，不能拿來代替電壓表。

上圖只是比較電路，不實作直接短接。若把電表切到電流檔，內部路徑不同，就不能照剛才的方式跨接電源。

換一支會切換電壓的程式

上週 GPIO5 是減一按鈕；現在先讓它成為輸出。

1. 移開表筆、拔 USB、電表 OFF。從開發板拔下外接線；麵包板的延長線可以留著。
2. 在 Arduino IDE 選 **File** → **Open**，開啟隨課程提供的 [GPIO 電壓程式](#)。完整程式也在本冊後方。
3. 找到 `PIN_TEST_OUTPUT = -1`。本稿示範接點是 **GPIO5**；只有課前已確認使用圖示板型與這個接點時，改為 5。未確認保留 -1，不猜其他數字。
4. 板子只插 COM 接頭 USB，依下表選設定，按左上方 **右箭頭 Upload**。

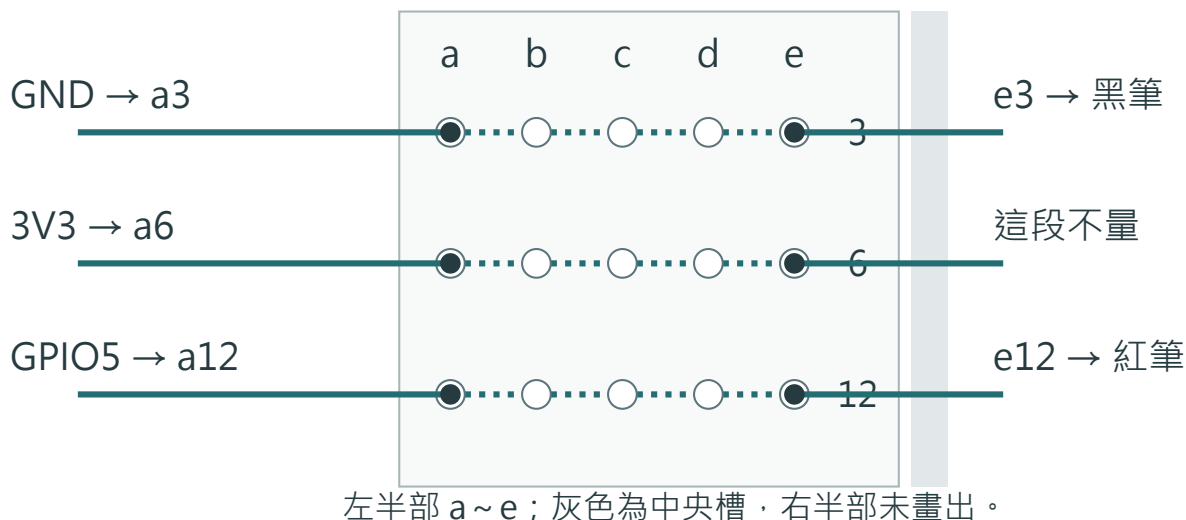
IDE 位置	本課圖示板設定
Tools → Board	ESP32S3 Dev Module
Tools → Port	插入此板後新增的 COM，號碼不固定
Flash Size / PSRAM	16MB / OPI PSRAM
USB CDC On Boot / USB Mode	Disabled / Hardware CDC and JTAG
Upload Mode / Upload Speed	UART0 / Hardware CDC / 115200
Flash Mode / Partition Scheme	QIO / 16M Flash (3MB APP/9.9MB FATFS)

按 **Tools** → **Serial Monitor**，速度選 **115200**。預期看到 `startup=LOW`，之後約每 10 秒出現 LOW 或 HIGH。出現 `gpio_profile_missing` 是腳位仍未填，不是電路壞了。

沿用 Week 2 的 IDE 安裝與 Espressif 套件。尚未安裝者依 Week 2 安裝段完成設定。上傳後先拔 USB，再依第 10 頁接 GPIO5 量測線。

加一條 GPIO5 線，量它的電壓

沒有按鈕、LED 或其他負載；GPIO5 只接電表量測。



起點 → 終點	現在的作用
GND → a3；e3 → 黑筆延長端	恢復共同參考點
3V3 → a6；e6 延長端分開收妥	恢復電源測點，這段不量它
GPIO5 → a12	新增程式控制的測點
e12 → 新的公對公延長線	紅筆稍後碰這個端點

每條外接線照表恢復，特別確認 GPIO5 不是 5Vin。第 12 列不能直接接到第 3 列或第 6 列。

第 12 列是 GPIO5 的量測接點。導線把 GPIO5 接到 a12，同列的延長線讓紅表筆量到這個腳位相對 GND 的電壓。

先確認舊按鈕已拆除。GPIO5 若輸出 HIGH 時仍被按鈕直接接 GND，會造成過大電流的風險。不要沿用計數器接線。

LOW、HIGH，電表真的不同嗎？

先猜：哪個階段接近 0 V？哪個接近 3.3 V？

1. 確認接線後插 COM USB，開 Serial Monitor，選 115200。
2. 電表選直流 20 V。黑筆碰 e3 的延長端；紅筆碰 e12 的延長端。
3. 看 phase=LOW 或 phase=HIGH，等讀值穩定再記錄。不要把切換瞬間的跳動值當穩態。

次數	phase	預期電壓	實測電壓
1	LOW	約 0 V	___ V
1	HIGH	約 3.3 V	___ V
2	LOW	約 0 V	___ V
2	HIGH	約 3.3 V	___ V
3	LOW	約 0 V	___ V
3	HIGH	約 3.3 V	___ V

再看一次啟動：移開兩支表筆，按一下板上的 RST，保存重新出現的 `startup=LOW` 文字。這裡只看啟動紀錄，不重做整張電壓表。

``expected_voltage=near_3.3V`` 是程式寫出的預期，不是 ESP32 已幫你量過。電表穩態讀值也不能證明 Reset 瞬間完全沒有短脈衝。

完成後拔 USB、電表 OFF。

week03_gpio_voltage_cycle.ino

第 1 ~ 33 行。

```
// Fill in only the GPIO value published in the verified course hardware profile.
// Keep -1 when no verified profile is available.
const int PIN_TEST_OUTPUT = -1;
const unsigned long PHASE_DURATION_MS = 10000;

bool outputHigh = false;
unsigned long phaseStartedMs = 0;
unsigned long cycleNumber = 1;

bool profileReady() {
    return PIN_TEST_OUTPUT >= 0;
}

void reportPhase() {
    Serial.printf(
        "cycle=%lu phase=%s expected_voltage=%s measure_now=true\n",
        cycleNumber,
        outputHigh ? "HIGH" : "LOW",
        outputHigh ? "near_3.3V" : "near_0V"
    );
}

void setup() {
    Serial.begin(115200);
    delay(500);

    if (!profileReady()) {
        Serial.println("week=3 status=blocked reason=gpio_profile_missing");
        return;
    }

    pinMode(PIN_TEST_OUTPUT, OUTPUT);
    digitalWrite(PIN_TEST_OUTPUT, LOW);
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week03_gpio_voltage_cycle.ino

第 34 ~ 56 行。

```
outputHigh = false;
phaseStartedMs = millis();

Serial.printf(
    "week=3 pin_test_output=%d startup=LOW phase_duration_ms=%lu\n",
    PIN_TEST_OUTPUT,
    PHASE_DURATION_MS
);
reportPhase();
}

void loop() {
    if (!profileReady()) return;

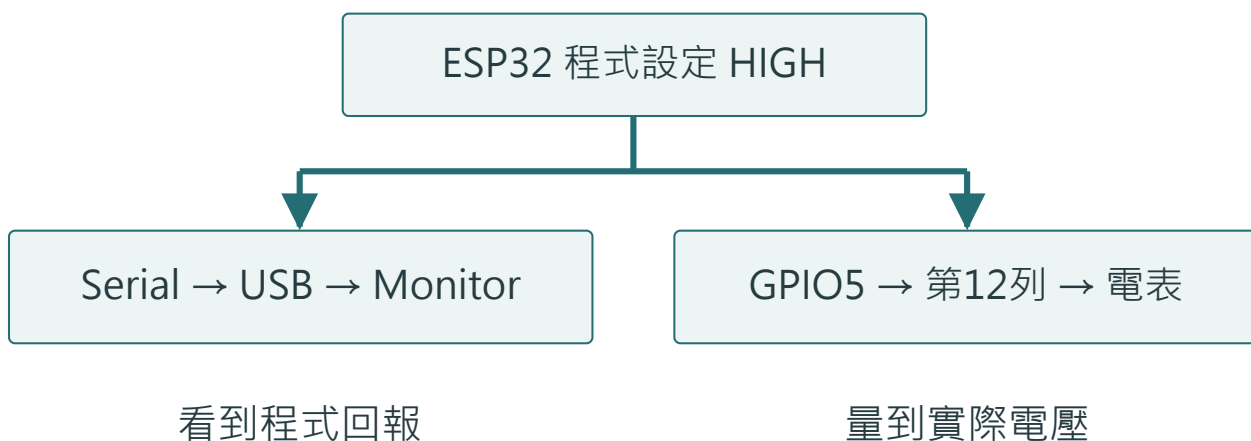
    unsigned long now = millis();
    if (now - phaseStartedMs < PHASE_DURATION_MS) return;

    phaseStartedMs = now;
    outputHigh = !outputHigh;
    digitalWrite(PIN_TEST_OUTPUT, outputHigh ? HIGH : LOW);
    if (!outputHigh) cycleNumber++;
    reportPhase();
}
```

同一份 .ino 檔案；分頁不代表另開程式。

一個命令，兩種證據

Serial 說「我要求 HIGH」；電表說「我量到幾伏特」。



程式中的動作	對應剛才哪件事
<code>pinMode(..., OUTPUT)</code>	指定 GPIO 用來輸出，不再讀按鈕
<code>digitalWrite(..., HIGH/LOW)</code>	要求輸出高 / 低準位
<code>millis()</code> 的時間差達 10000	約等 10 秒才切換一次
<code>Serial.printf(...)</code>	把命令與次數送到電腦

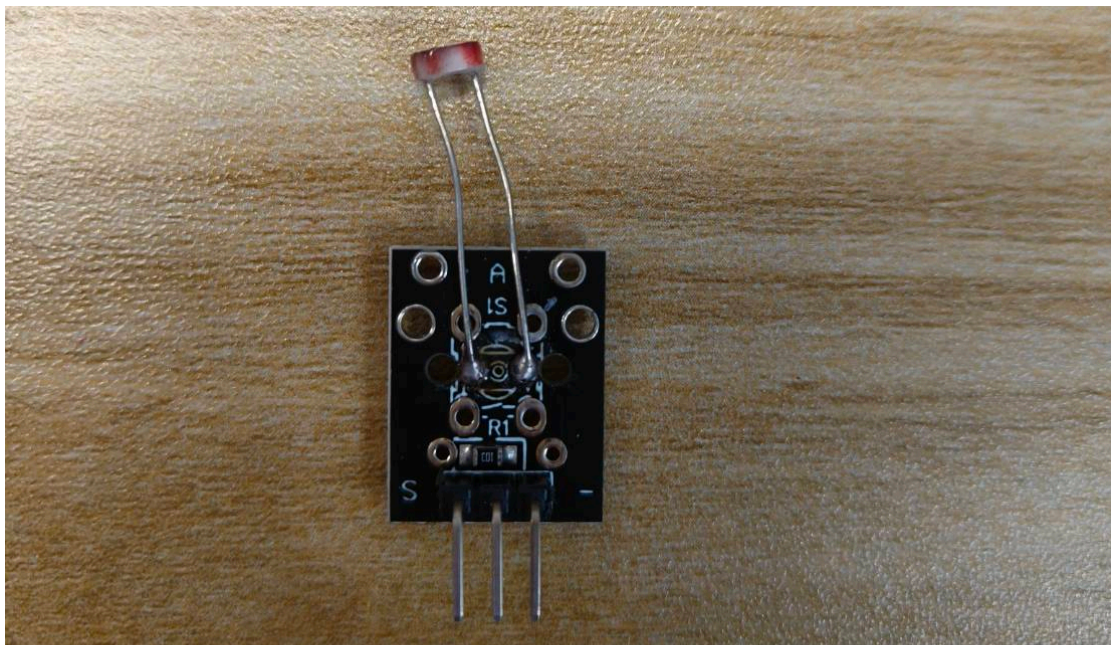
3V3 和 GPIO5 不相同。3V3 是供電腳；GPIO5 是訊號腳，電壓由程式控制。讀到約 3.3 V，不代表 GPIO 可以當一般 3.3 V 電源帶動馬達或模組。

自己回答：如果 Monitor 顯示 HIGH，電表卻是 0 V，能說程式完全正確、接線也正確嗎？先斷電查腳位、列號和上傳版本，不能只相信文字。

下一段要讀光敏訊號。保持斷電，拆除 GPIO5 → a12 與 e12 延長線；3V3、GND 兩列稍後繼續使用。

找到會感光的元件和三支腳

模組先放桌上，不插麵包板，不接 USB。



實物照片局部放大，原圖未修改：圓形感光元件朝上，三支排針朝下。

照片中的排針	已有實物紀錄的用途
左邊，獨立 S 標示旁	訊號，稍後量它的電壓
中間，沒有獨立供電標示	本課已核對模組的供電端
右邊，獨立 — 標示旁	接 GND

比對自己的絲印和照片。板上 **A**、**S1**、**R1** 是元件區標示，不能直接當排針名稱。三組路徑須在斷電下量測。

照片朝向改了，左右也會改。不同模組不能只照左右猜供電。本稿只適用已確認「中間到 S 是固定電阻、S 到 — 是光敏電阻」的接法；結果不符就不供電。

不接電，也能先看阻值變化

這裡改用 Ω 檔；ESP32、USB、電池都不能接在模組上。

Ω 是電阻單位「歐姆」；k 表示一千，所以 $1\text{ k}\Omega = 1000\ \Omega$ 。200k Ω 是量程，不是模組一定有 200 k Ω 。

1. 黑筆仍插 COM，紅筆仍插 V Ω mA。先選 **200k Ω** ，不是直流 20 V。
2. 黑、紅筆各碰指定排針，手只握絕緣部分。固定接觸後，用紙遮住感光面，不碰動排針。
3. 同一組測點依序比較「室內光 → 遮光 → 移開紙」。換測點時也保持模組不接電源。

測哪兩腳	室內光	遮光	移開紙
中間 ↔ S	___ k Ω	___ k Ω	___ k Ω
S ↔ -	___ k Ω	___ k Ω	___ k Ω
中間 ↔ -	___ k Ω	___ k Ω	___ k Ω

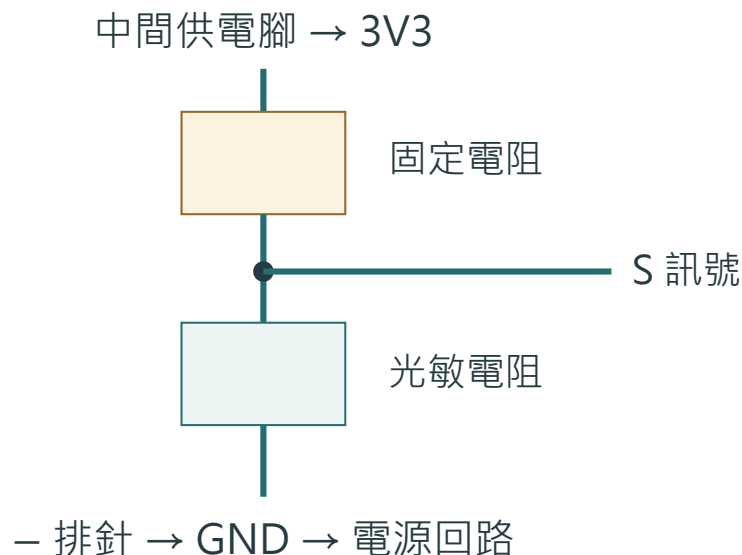
記下**檔位、完整小數點與單位**。200k 檔例如顯示 10.0，通常是 10.0 k Ω ，仍以自己的表型說明為準。若超出量程顯示 1 / OL，不要寫成 1 Ω ；提高量程或保存「超量程」。

先看變化，不求精密驗證：本課既有接法預期中間 ↔ S 比較固定；S ↔ - 隨光線改變。沒有變化先查遮光與接觸，不交換供電腳來試。

量完電表 OFF，移開表筆。

光敏模組裡有兩個電阻

剛才量的三組腳，是兩段電阻及它們的總路徑。



上面的固定電阻：接在中間供電腳與 S 之間。**下面的光敏電阻：**接在 S 與 — 之間。S 是兩者中間的位置，不是新的電源。

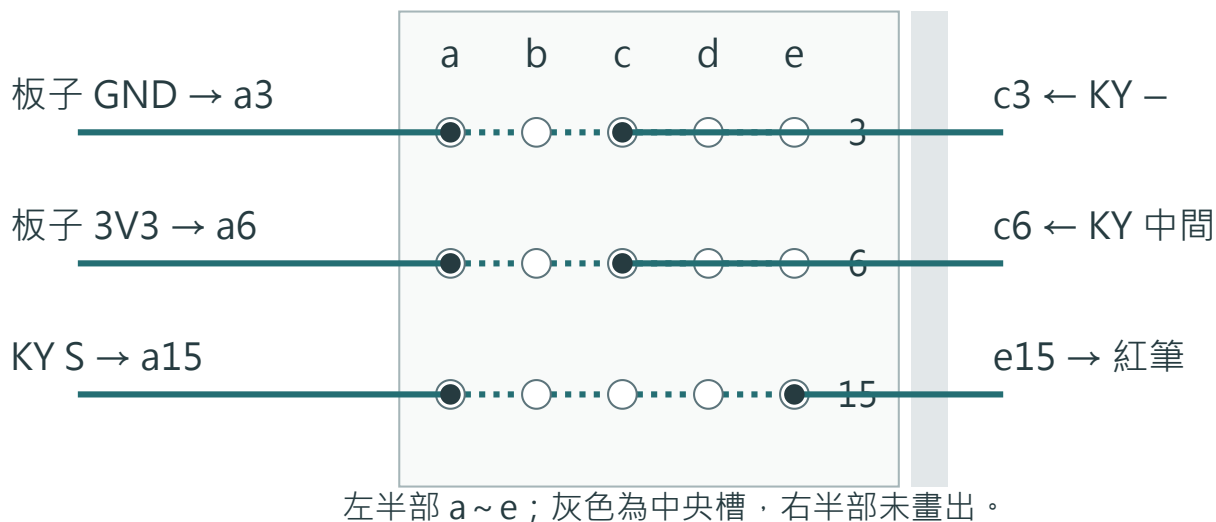
剛才量的路徑	這張圖的對應
中間 ↔ S	上面一段
S ↔ —	下面一段
中間 ↔ —	兩段串聯的總路徑

同樣光照與接觸下，總阻值應大致接近兩段相加。三次量測若光照變了、手碰到金屬或量程不同，不能把差異當成精密驗證通過。

S 相對 GND 的電壓取決於兩段電阻的比例。光敏電阻改變時，這個中間位置的電壓也會改變。

先接供電，再把 S 引出來

這一步只給模組供電。S 還不接 GPIO4。



起點 → 終點	說明
ESP32 GND → a3 ; 3V3 → a6	保留前段兩條電源線
KY 中間供電腳 → c6	用母頭套模組腳，公頭插孔
KY - → c3	接到同一個 GND
KY S → a15	只是引出訊號，不接 GPIO
e15 → 新的公對公延長線	紅表筆的訊號測點
e3 → 黑筆延長線	沿用 GND 測點

GPIO5 與第 12 列的線已拆除。e6 電源延長線這段不用，斷電時拆下，避免紅筆量錯。模組放不導電桌面，不把三支腳插在麵包板同一個五孔組。

一起核對中間腳、S、- 及列號。S 不直接短接 GND；GPIO4 暫時空著。只用 3V3，不用 5Vin、5V 或其他電池供電。

遮住之後，電壓升還是降？

先猜一次，再用自己的讀值回答。

1. 電表選 **直流 20 V**，不能留在上一段的 Ω 檔。
2. 接 COM USB。黑筆碰 e3 延長端；紅筆碰 **e15** 延長端，不是 e6。
3. 不移動模組，記錄室內光；用不透光紙遮住感光面，再記錄；移開紙，最後再記一次。

條件	預期觀察	實測 S 對 GND
室內光	可穩定讀取一個電壓	___ V
原位置遮光	電壓與室內光有差異	___ V
移開紙	接近同位置室內光讀值	___ V

我的觀察：遮光後電壓 _ _ ；移開紙後 _ _ 。板子、檯燈、紙的位置與表筆接觸是否都固定？
_ _ 。

不憑 KY-018 商品名稱判斷方向。對圖示電路，下方光敏電阻變大時，S 電壓會升高；若實測相反，先保留結果，斷電核對模組內部接法與測點。

讀值超出 0 ~ 3.3 V、負值或異常發熱：先移開表筆、斷電，查供電與測點。正常觀察完成後也先移開表筆、拔 USB、電表 OFF。

如果 S 一直等於電源電壓，移開表筆並確認量的是 e15，不是 e6；接點鬆脫要斷電後處理。

把 S 的電壓變成電腦上的數字

先換程式，再把訊號接入 GPIO4。不要沿用 GPIO 輸出程式。

1. 保持斷電，從開發板拔下外接線。麵包板與模組間的線可保留，沒有電源供應。
2. IDE 選 **File** → **Open**，開啟 [光敏原始讀值程式](#)。
3. 找 `PIN_LIGHT = -1`。課前已確認本稿的 GPIO4 接法時改成 4，否則保留 -1。不要沿用上一支程式的 5。
4. 將 `DEVICE_ID = "CHANGE_ME"` 改成自己的組別代碼，例如 "G01"。不要放姓名或學號。
5. 板子只接 COM USB，依第 9 頁重新核對板型與 Port，按 Upload。

Serial Monitor 選 115200。看到 `status=ready` 及 `interval_ms=500`，表示程式已開始準備取樣，不代表線已接對。

```
device=G01 ... adc_bits=12 interval_ms=500
```

這是欄位示意，不是本次實測。尚未接模組的輸入可能亂跳，**現在的 raw 不拿來當光線資料**。

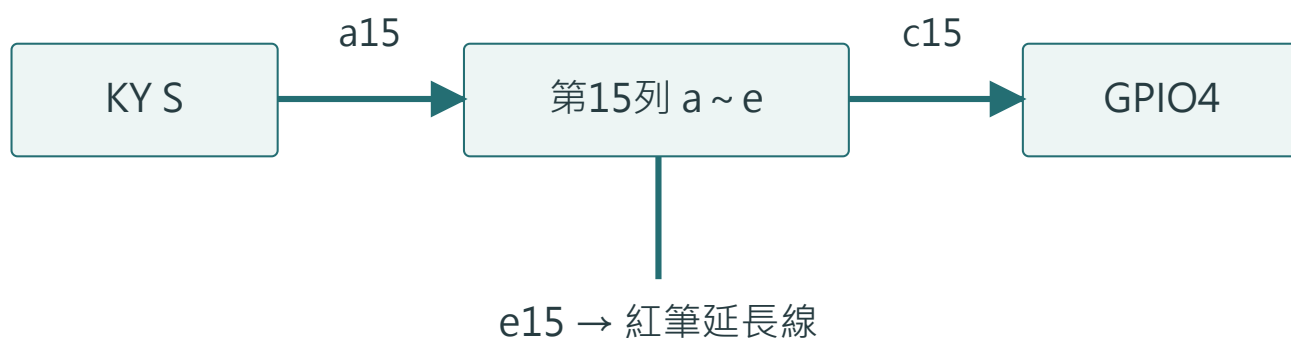
ADC 是晶片裡把電壓轉成數字的功能。`analogRead` 取得的是原始整數，不是已換算的伏特。

完成後拔 USB，依第 21 頁接回模組電源與 S 訊號。

多一條線：c15 接到 GPIO4

第 15 列是同一個 S 訊號，電表和 ESP32 可以看同一位置。

新增訊號線；模組供電接法保持不變



黑筆 → e3 / GND；模組 — → c3 / GND。

核對這些起點 → 終點	現在要做
ESP32 GND → a3；3V3 → a6	接回剛才拔下的電源線
KY — → c3；中間 → c6；S → a15	維持上一段接法
c15 → ESP32 GPIO4	新增公對母線；不是 GPIO5
e15、e3 的兩條延長線	分開固定，供稍後量 S / GND

a15、c15、e15 是同一五孔組的三個入口。不是三個不同訊號，也不需要再把 S 直接接一條線到 GND。

GPIO5 保持空著；4 不是 3V3。所有訊號只能在已確認的安全電壓內。ESP32-S3 GPIO 不接受 5 V 訊號；本週模組以 3V3 供電。

對照接點後插 COM USB，開 Monitor，選 115200。先固定模組、遮住、移開一次，看數字是否改變，再開始保存資料。

Monitor 的文字不會自動存檔

保持接線與感測器位置不變。室內光與遮光各存十筆，分成兩份檔案；檔名註明條件，不把試打文字混入量測資料。

1. Serial Monitor 選 115200，等數行完整文字出現。資料仍會繼續來，不用拔 USB 暫停。
2. 在 Monitor 關閉自動捲動，或捲回要保存的位置。用滑鼠從第一行開頭拖到最後一行結尾，按 **Ctrl+C**。
3. 開 Windows「記事本」，按 **Ctrl+V**。確認每行有完整的 `sample`、`raw` 和時間，不是只有截圖或最後半行。
4. 最上方加一行條件，例如「日期 / group01 / 室內光 / 同一桌面 / 剛才未重開機」。
5. 選「檔案 → 另存新檔」，檔名例如 `week3_group01_indoor.txt`，編碼選 **UTF-8**。回 Monitor 繼續觀察。

兩份量測檔案	應有的內容
<code>week3_group01_indoor.txt</code>	同輪室內光十筆完整行與條件
<code>week3_group01_shade.txt</code>	同接法遮光十筆完整行與條件

若拖選漏行，就重新選取那一段；不要補寫不存在的輸出。重開機或換位置後另存一輪，不能把不同輪資料混在一起。後面各週也用同樣方法保存 **Serial** 紀錄。

室內光十筆，遮光十筆

不是只挑最好看的兩個數字，也不是把不同位置的資料拼起來。

1. 模組、桌燈、電源、接線位置不變。先預測遮光後 raw 比室內光大還是小：_____。
2. 室內光穩定後，從 Monitor 保存連續 **10 行**，標記「室內光」。保留序號、時間及 raw。
3. 用同一張紙在原位置遮住感光面。穩定後另存連續 **10 行**，標記「遮光」。
4. 每種條件也量一次 S 對 GND：直流 20 V，紅 e15、黑 e3。先量再存 log，不宣稱兩者精確同步。不要帶電移線。

```
device=G01 sample=21 uptime_ms=10500 light_raw=310
device=G01 sample=22 uptime_ms=11000 light_raw=314
```

上面是**預期格式示例**，**不是實測**。只讀最後的 light_raw 作光線原始值；sample 是序號，uptime_ms 是開機後毫秒。

條件	10 筆最小值	10 筆最大值	電表
室內光	_____	_____	_____ V
遮光	_____	_____	_____ V

保存全部 **20 行**，表格只是摘要。500 ms 是 0.5 秒；10 筆有 9 段間隔，首尾約 4.5 秒。若中途重開機或移位置，另開一輪，不混算。

若一直顯示 0 或 4095，先用直流電壓檔量 e15 對 e3，再斷電查線。端點可能來自飽和或接線問題，不能只憑 raw 判斷斷線。

week03_ky018_raw.ino

第 1 ~ 32 行。

```
// Fill in only the ADC GPIO published in the verified course hardware profile.
// Keep -1 when no verified profile is available.
const int PIN_LIGHT = -1;
const unsigned long SAMPLE_INTERVAL_MS = 500;
const char* DEVICE_ID = "CHANGE_ME";

unsigned long lastSampleMs = 0;
unsigned long sampleNumber = 0;

bool profileReady() {
    return PIN_LIGHT >= 0;
}

void setup() {
    Serial.begin(115200);
    delay(500);

    if (!profileReady()) {
        Serial.println("week=3 sensor=ky018 status=blocked reason=gpio_profile_missing");
        return;
    }

    analogReadResolution(12);
    analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
    Serial.printf(
        "week=3 sensor=ky018 status=ready device=%s pin_light=%d "
        "adc_bits=12 interval_ms=%lu\n",
        DEVICE_ID,
        PIN_LIGHT,
        SAMPLE_INTERVAL_MS
    );
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week03_ky018_raw.ino

第 33 ~ 50 行。

```
void loop() {
  if (!profileReady()) return;

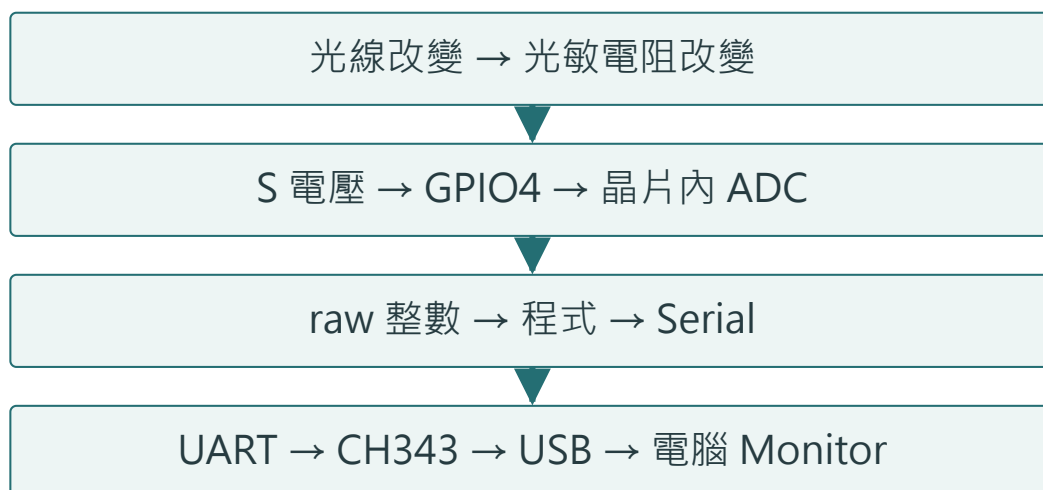
  unsigned long now = millis();
  if (now - lastSampleMs < SAMPLE_INTERVAL_MS) return;
  lastSampleMs = now;
  sampleNumber++;

  int lightRaw = analogRead(PIN_LIGHT);
  Serial.printf(
    "device=%s sample=%lu uptime_ms=%lu light_raw=%d\n",
    DEVICE_ID,
    sampleNumber,
    now,
    lightRaw
  );
}
```

同一份 .ino 檔案；分頁不代表另開程式。

raw 不是伏特，也不是亮度單位

電表顯示 V；程式目前顯示的是 ADC 原始整數。



圖中的 UART 是 ESP32 傳送這些文字的通訊介面；CH343 是圖示板 COM 接頭使用的 USB 轉接晶片。本頁是資訊路徑，不是新增接線指令。

12-bit 在這支程式中代表 **4096 個編號**，從 0 到 4095。raw=700 不是 700 V、0.700 V，也不是 700 lux。

程式設定 ADC_11db；Espressif 文件列 ESP32-S3 此設定的可量測範圍約 0~3.1 V。靠近端點可能飽和，不能假設 4095 正好等於 3.3 V，也不能保證每一格都同樣精準。

現在可以說	不能直接說
此輪遮光後，我的 raw 上升 / 下降	所有 KY-018 都同方向
電表與 raw 在此輪大致同向改變	我已精確校正 ADC 電壓
接法相同時，數字可用來比較	數字就是照度 lux

一起解釋：用自己的 20 行資料回答「raw 變大代表比較亮嗎？」先說哪個條件，再指出兩組範圍，不能只回答是或不是。

模組 — 與板子 GND 是共同參考；訊號送到 GPIO4。ADC 內部完成比較，不是叫你在板外用線把 GPIO4 和 GND 短接。

光敏模組有電，不等於 ADC 有訊號

資訊處理示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想： 保留模組的 3V3 與 GND，只在圖上斷開 S 到 GPIO 的線。raw 會一定變成 0 嗎？

原理提示： 模組可能仍正常供電，但 ADC 腳失去可靠訊號；讀值不能當光線資料，也不保證是 0。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0、LOW 或「一定停止」。

先畫兩段範圍，再選分界

用剛才資料，不重新收一樣的二十筆。

教學假資料；raw 無單位，不是 V



示意位置非等比例。兩段之間的值沒有基準資料。

上圖是**教學假資料**：室內光 300 ~ 320，遮光 900 ~ 920。兩段沒有重疊，中間可以選一個分界：

分界 = (室內最大值 320 + 遮光最小值 900) ÷ 2 = 610。在這個方向下，raw ≤ 610 判室內光，raw > 610 判遮光。

你的資料狀態	接下來怎麼做
遮光整段比室內光大	用室內最大值與遮光最小值求中點
遮光整段比室內光小	用遮光最大值與室內最小值求中點；方向反過來
重疊，或兩段碰到同一數字	目前無法用這個分界分清，先保存結果，不硬設

填自己的四個值：室內最小 ____、室內最大 ____、遮光最小 ____、遮光最大 ____。我的分界 ____；遮光是較大 / 較小 / 還無法分開。

最小 / 最大值是這一輪的範圍，不是未來永遠的界線。不要刪掉不合預期的數字，只為了讓範圍分開。

讓程式印出「室內光 / 遮光」

這次 GPIO4 仍是輸入，模組接法不變。

先移開表筆、拔 USB、電表 OFF，從開發板拔下外接線。關閉上一個 Monitor，IDE 開啟 [光線分類程式](#)，只填下面這些設定：

程式名稱	填什麼
PIN_LIGHT	與上一支原始讀值程式相同、已確認的腳位
DEVICE_ID	同一組別代碼
INDOOR_MIN、INDOOR_MAX	自己室內光十筆的最小、最大值
SHADE_MIN、SHADE_MAX	自己遮光十筆的最小、最大值

分界與方向由程式算，不必另把 610 抄進程式。板子只接 COM USB，依第 9 頁設定後 Upload。拔 USB，依第 21 頁接回光敏電路；再接 USB，Monitor 選 115200。

```
status=ready ... threshold=610 shade_higher=true
... light_raw=310 ... class=INDOOR label=室內光
```

上面沿用前頁假資料示範格式，**你的分界應來自自己的資料**。若出現 `calibration_missing_or_overlap`，表示有缺值、不合範圍或重疊；回前頁查四個設定，不拆線試錯。

既有分類程式保守排除 `raw=0、4095` 的基準。這是課堂規則，不代表端點讀值一定是斷線。尚未取得可分開的基準時，保留原始資料與原因就好，不偽造通過。

規則設好了，不代表一定判對

不用剛才建立規則的二十筆，改看接下來的新資料。

1. 先在相同位置回到室內光，保存 5 筆新紀錄。
2. 再以相同方式遮光，保存 5 筆新紀錄。
3. 每筆比較「你實際做的動作」與 label，也保留 quality、reason。

真實操作條件	新資料筆數	標籤相符	不相符 / 未判定
室內光	5	——	——
遮光	5	——	——

例：若 10 筆有 8 筆標籤相符，可以說「這次、這個位置的 10 筆有 8 筆相符」。不能說整個系統準確率永久 80%，也不能把短時間連續取樣視為十種環境。

若全部都判同一類：先對照原始值與四個範圍，確認條件、上傳版本、模組位置與組別資料。不要先交換電源線。

同一個 raw 落在兩組範圍之間時，程式雖然印了標籤，基準紀錄中是否真的出現過這個數字？

week03_light_classifier.ino

第 1 ~ 31 行。

```
// Relative light classification: calibration ranges come from the student's log.
// No pin is approved by compiling this program. Keep -1 until a profile is published.
#include <Arduino.h>

const int PIN_LIGHT = -1;
const char *DEVICE_ID = "CHANGE_ME";
const int INDOOR_MIN = -1;
const int INDOOR_MAX = -1;
const int SHADE_MIN = -1;
const int SHADE_MAX = -1;
const unsigned long SAMPLE_INTERVAL_MS = 500;
bool ready = false;
bool shadeHigher = true;
int threshold = -1;
unsigned long lastSampleMs = 0;
unsigned long sample = 0;

bool configureCalibration() {
    // End-point values are excluded by this classroom policy, not proof of a fault.
    if (INDOOR_MIN <= 0 || SHADE_MIN <= 0 || INDOOR_MAX >= 4095 ||
        SHADE_MAX >= 4095 || INDOOR_MIN > INDOOR_MAX || SHADE_MIN > SHADE_MAX)
        return false;
    if (INDOOR_MAX < SHADE_MIN) {
        shadeHigher = true;
        threshold = (INDOOR_MAX + SHADE_MIN) / 2;
    } else if (SHADE_MAX < INDOOR_MIN) {
        shadeHigher = false;
        threshold = (SHADE_MAX + INDOOR_MIN) / 2;
    } else return false; // Overlap or touching ranges cannot be separated here.
    return true;
}
```

同一份 .ino 檔案；分頁不代表另開程式。

week03_light_classifier.ino

第 32 ~ 63 行。

```
const char *classifyLight(int raw) {
    if (!ready || raw <= 0 || raw >= 4095) return "UNDECIDED";
    const bool lowSide = raw <= threshold;
    return (shadeHigher ? !lowSide : lowSide) ? "SHADE" : "INDOOR";
}

const char *chineseLabel(const char *state) {
    if (strcmp(state, "SHADE") == 0) return "遮光";
    if (strcmp(state, "INDOOR") == 0) return "室內光";
    return "未判定";
}

const char *qualityReason(int raw) {
    if (raw <= 0 || raw >= 4095) return "adc_endpoint";
    const int observedMin = min(INDOOR_MIN, SHADE_MIN);
    const int observedMax = max(INDOOR_MAX, SHADE_MAX);
    if (raw < observedMin || raw > observedMax) return "outside_observed_span";
    const bool inIndoor = raw >= INDOOR_MIN && raw <= INDOOR_MAX;
    const bool inShade = raw >= SHADE_MIN && raw <= SHADE_MAX;
    if (!inIndoor && !inShade) return "between_baselines";
    return "within_baseline_range";
}

void setup() {
    Serial.begin(115200);
    delay(500); // Only startup; sampling below does not block for half a second.
    if (PIN_LIGHT < 0) {
        Serial.println("week=3 status=blocked reason=gpio_profile_missing");
        return;
    }
    if (!configureCalibration()) {
```

同一份 .ino 檔案；分頁不代表另開程式。

week03_light_classifier.ino

第 64 ~ 88 行。

```
    Serial.println("week=3 status=blocked reason=calibration_missing_or_overlap");
    return;
}
analogReadResolution(12);
analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
ready = true;
lastSampleMs = millis();
Serial.printf("week=3 status=ready device=%s threshold=%d shade_higher=%s
interval_ms=%lu scope=local_only\n",
              DEVICE_ID, threshold, shadeHigher ? "true" : "false",
              SAMPLE_INTERVAL_MS);
}

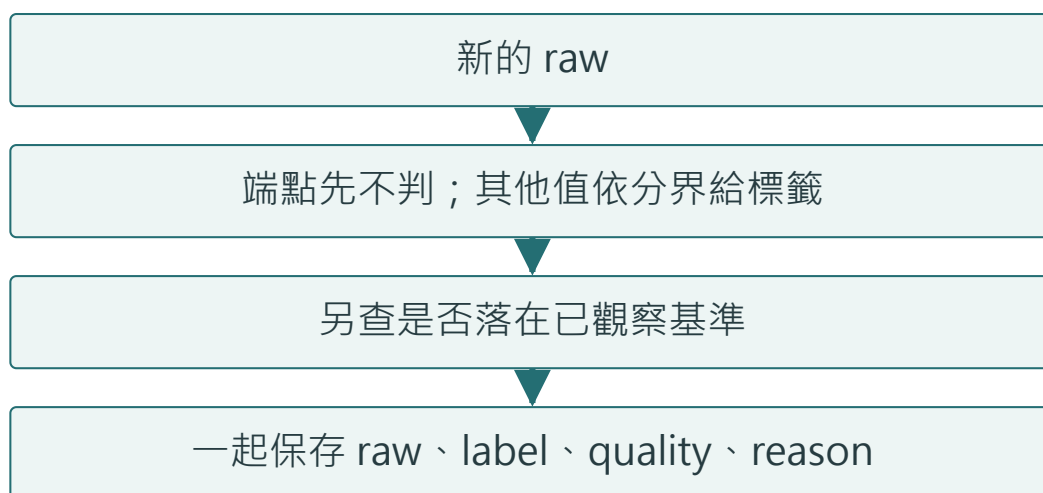
void loop() {
    if (!ready) return;
    const unsigned long now = millis();
    if (now - lastSampleMs < SAMPLE_INTERVAL_MS) return;
    lastSampleMs = now; // Late iterations take one current sample, not fake catch-up
    samples.
    const int raw = analogRead(PIN_LIGHT);
    const char *state = classifyLight(raw);
    const char *reason = qualityReason(raw);
    const bool plausible = raw > 0 && raw < 4095;
    Serial.printf("device=%s sensor=ky018 source=hardware sample=%lu uptime_ms=%lu
light_raw=%d threshold=%d class=%s label=%s valid=%s quality=%s reason=%s\n",
                  DEVICE_ID, ++sample, now, raw, threshold, state, chineseLabel(state),
                  plausible ? "true" : "false",
                  strcmp(reason, "within_baseline_range") == 0 ? "relative_only" :
"suspect", reason);
}
```

同一份 .ino 檔案；分頁不代表另開程式。

看 raw，也看為什麼這樣判

沿用假資料：室內 300 ~ 320、遮光 900 ~ 920、分界 610。

新 raw	程式標籤	quality / reason 的意思
310	室內光	relative_only：在已觀察的基準內，仍只是本地相對判斷
700	遮光	suspect / between_baselines：落在兩段之間，沒有基準支持
1000	遮光	suspect / outside_observed_span：超出曾觀察的總範圍
0 或 4095	未判定	suspect / adc_endpoint：端點先不分類，不等於證明斷線



這些英文是程式輸出的欄位值：raw 是原始數字，label 是規則給的文字，reason 是這筆數字和基準的關係。它們不能互相替代。

討論：假設剛才遮光較大，現在換位置卻較小，先查哪兩件事？先保存兩輪，不合併；再查接法 / 模組是否相同，以及光線、遮法和接觸是否改變。找到差異前不能套用舊規則。

下一段斷電後拆下光敏模組。用兩顆固定電阻，把「為什麼電壓會變」拆成看得見的電路。

拿出 1 k Ω 和 10 k Ω

先量實物，再接電路。不是只看包裝就假設一定正確。

移開表筆，拔 USB，電表 OFF。拆下 KY 的供電、GND、S 線，以及 GPIO4 線與第 15 列延長線。保留 GND $\rightarrow$ a3、3V3 $\rightarrow$ a6；板子仍斷電。



已購電阻包的實物與手寫標示。本段只取 1 k Ω 、10 k Ω 各一顆。

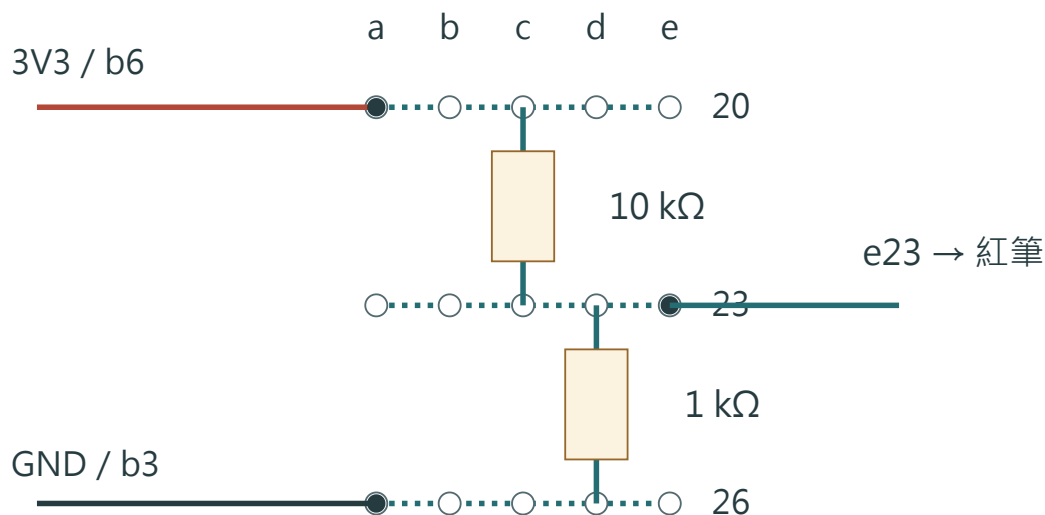
1. 每次拿一顆，與其他電路完全分離。
2. 黑筆 COM、紅筆 V Ω mA，選 20k Ω 。兩筆各碰電阻一端，不同時用手捏兩端金屬。
3. 記錄後電表 OFF。電阻不分正負，但接到電路的上下位置等一下會影響輸出。

包裝標示	完整畫面	換算後的阻值
1 k Ω	___	___ k Ω = ___ Ω
10 k Ω	___	___ k Ω = ___ Ω

1 k Ω = 1000 Ω 。理想 1 k Ω 在 20k 檔約顯示 1.00；理想 10 k Ω 約顯示 10.00。這是判讀示例，不是要求你的數值完全一樣。

先把 10 kΩ 放上面

上、下是電路的位置：靠 3V3 是上，靠 GND 是下。



黑筆仍碰 e3 延長端；電阻兩端跨不同列。

元件 / 線	兩端位置
供電延長線	b6 → a20
10 kΩ	c20 → c23
1 kΩ	d23 → d26
接地延長線	a26 → b3
紅筆用公對公延長線	e23 → 自由端
黑筆延長線	e3 → 自由端

第 23 列讓兩顆電阻接在一起。電阻的兩隻腳不能插進同一五孔組；上下端不能另加導線跨過電阻。輕彎腳配合孔距，不硬拉元件本體。

依表核對，確認 KY、GPIO4、GPIO5 都沒有接線。中間第 23 列會更接近 0 V，還是 3.3 V？寫下預測 ____ V，再依量測步驟比較。

量到之後，再算一次

這裡只量中間電壓，不使用 ADC，也不量電流。

確認接法後插 USB。電表選直流 20 V，黑筆碰 e3 延長端，紅筆碰 e23 延長端。我的結果：
_____ V。保存接線照片後移開表筆。

為什麼預期約 0.3 V？ 電流先經上面 10 kΩ，再經下面 1 kΩ，最後回到 GND。兩顆串聯，流過它們的電流相同；比較大的電阻分到比較大的電壓差。

供電電壓是 3V3 對 GND；中間電壓是第 23 列對 GND；上方電阻靠近 3V3，下方電阻靠近 GND。電壓單位 V，兩個電阻須使用相同單位。

$$\begin{aligned}\text{中間電壓} &= \text{供電電壓} \times \text{下方電阻} \div (\text{上方電阻} + \text{下方電阻}) \\ &= 3.3 \text{ V} \times 1 \text{ k}\Omega \div (10 \text{ k}\Omega + 1 \text{ k}\Omega) \\ &= 0.3 \text{ V}\end{aligned}$$

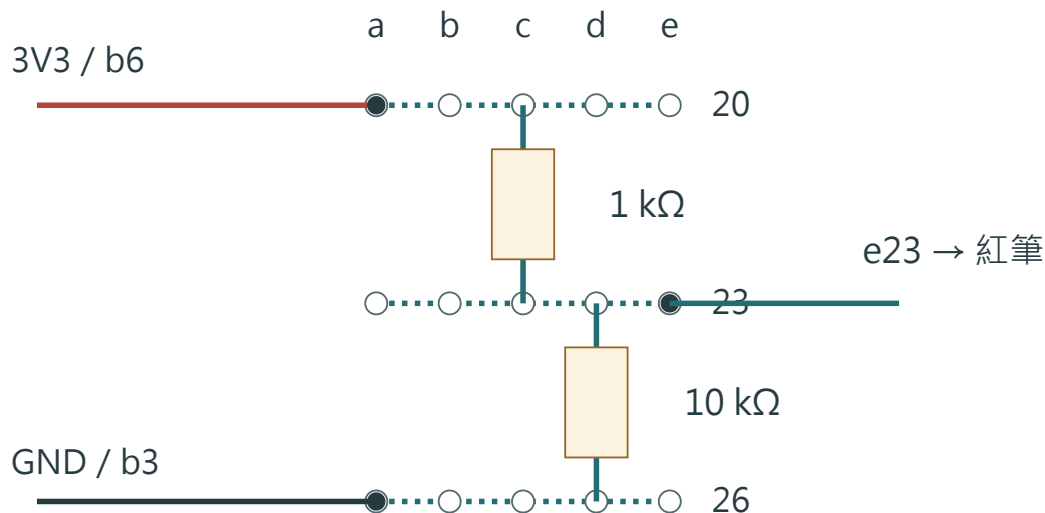
位置	理想電位	該電阻兩端差值
供電端 → 中間	3.3 V → 0.3 V	上方電阻：3.0 V
中間 → GND	0.3 V → 0 V	下方電阻：0.3 V

電表量的是中間到 GND，所以對應**下方電阻的電壓差**。若實際供電不是 3.3 V，用自己量的供電電壓與阻值重算，不把差異直接當成錯接。

交換電阻前，先拔 USB、確認 PWR 熄滅、電表 OFF。

交換位置，總電阻明明沒變？

原線不動，只交換兩顆電阻的上下位置。



黑筆仍碰 e3 延長端；電阻兩端跨不同列。

USB 保持拔除：把 $1\text{ k}\Omega$ 接 $c20 \rightarrow c23$ ； $10\text{ k}\Omega$ 接 $d23 \rightarrow d26$ 。其餘線沿用前頁。先預測中間電壓：_____ V。

核對後插 USB，直流 20 V，黑 e3、紅 e23。我的結果：_____ V。保存第二張接線照片，再移開表筆、拔 USB、電表 OFF。

$$\begin{aligned}\text{中間電壓} &= 3.3\text{ V} \times 10\text{ k}\Omega \div (1\text{ k}\Omega + 10\text{ k}\Omega) \\ &= 3.0\text{ V}\end{aligned}$$

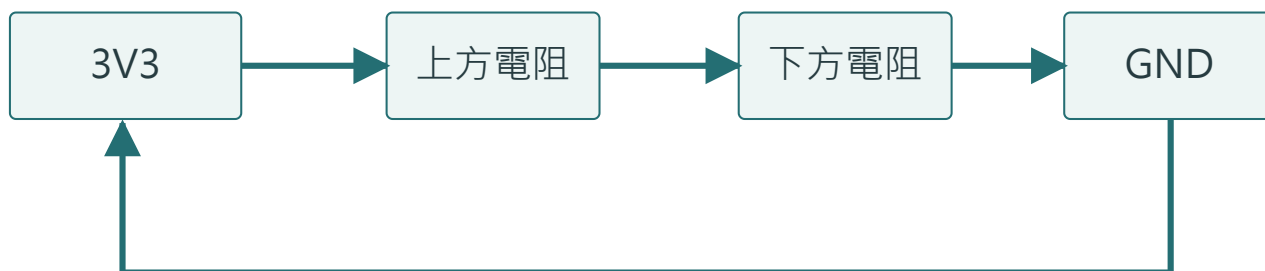
原來 $10\text{ k}\Omega$ 不是「固定扣掉幾伏特」。兩顆總阻值仍是 $11\text{ k}\Omega$ ；中間到 GND 現在跨的是 $10\text{ k}\Omega$ ，所以中間電壓不同。

回看 KY 模組：上方固定電阻不變，下方光敏電阻隨光改變，中間 S 的電壓就會跟著變。另一種內部接法可能方向相反，不能只憑商品名稱判方向。

同一條路，電流不會先被用掉

電阻限制整條串聯路徑的電流，不是把 3.3 A 變成小一點。

兩顆串聯電阻流過相同電流



經板上電源回路返回；畫的是傳統電流方向。

I 是電流，單位 A； V 是電壓差，單位 V； R 是電阻，單位 Ω 。歐姆定律在這個電阻電路中寫成 $I = V \div R$ ，不是 $V \times R$ 。

$$\begin{aligned} I &= 3.3 \text{ V} \div (10000 \Omega + 1000 \Omega) \\ &= 0.0003 \text{ A} = 0.3 \text{ mA} \end{aligned}$$

這是理想計算，不是電表實測電流。交換上下位置，總阻值不變，所以理想總電流仍相同。

紙上比較，不實際新增接線	判斷
3V3 → 兩顆串聯電阻 → GND	有控制電流的阻值，是本次正常電路
3V3 → 普通導線 → GND	電源被低阻路徑直接接通，這就是短路
路徑中拔掉一端	斷路，這條路不再形成完整回路

短路**可能**造成發熱、保護動作或損壞；不代表每次一定燒壞。沒有外加電阻也不等於 $R=1 \Omega$ ，導線電阻可能遠小於 1Ω ，實際電流還受電源與接觸限制。

分壓器少一顆電阻，接點電壓呢？

電路連接示意；僅作圖上推演，保持現有實物接線不動。

正常的連接 / 處理



只改標記的地方



線表示電路連接；不是實物腳序或完整接線圖。

想一想：只拆掉下方 $1\text{ k}\Omega$ ，留下斷路，不用線補上。紅筆仍量中間接點、黑筆仍接 GND，讀值會接近 0 V ，還是 3.3 V ？

原理提示：高阻抗量測下，上方電阻電流變得很小，壓降接近 0 ；中間接點反而接近 3.3 V 。

只有標記處改變，其餘供電、程式與環境條件沿用正常情境。不可把未知結果直接寫成 0 、LOW 或「一定停止」。

按一下，才記下一筆光線

用按鈕決定何時保存感測值。

固定光敏模組的位置，比較紙盒打開、蓋上時的讀值。這裡記錄光線變化，不宣稱能自動辨認所有盒子。

零件或工具	這次的作用
按鈕接 GPIO5	觸發一次記錄
光敏 S 接 GPIO4	提供當下的原始讀值
Serial Monitor	顯示序號、raw 與開機時間

預期結果：按一下印一筆；持續按住不重複印；放開再按，才新增下一筆。這需要程式記住按鈕是否已放開，不能只用「讀到 LOW 就記錄」。

Week2 的按鈕接線與 HIGH / LOW 概念沿用；本例另外加入按壓辨識，做法與完整程式都列在本冊。

GPIO5 這次只讀按鈕

本週前面 GPIO5 曾經輸出電壓；不能直接沿用那支程式接按鈕。

1. 拔 USB、拆掉全部外接線；收起分壓電阻與 GPIO5 電壓量測線。
2. 開啟 [button_light_capture.ino](#)。依下表填入已確認設定，Ctrl+S 儲存；依第 9 頁設定，只接 COM USB 上傳。
3. Monitor 選 115200，應看到 event=ready release_button_first。關閉 Monitor、拔 USB，才依表重接。不要接 5Vin。

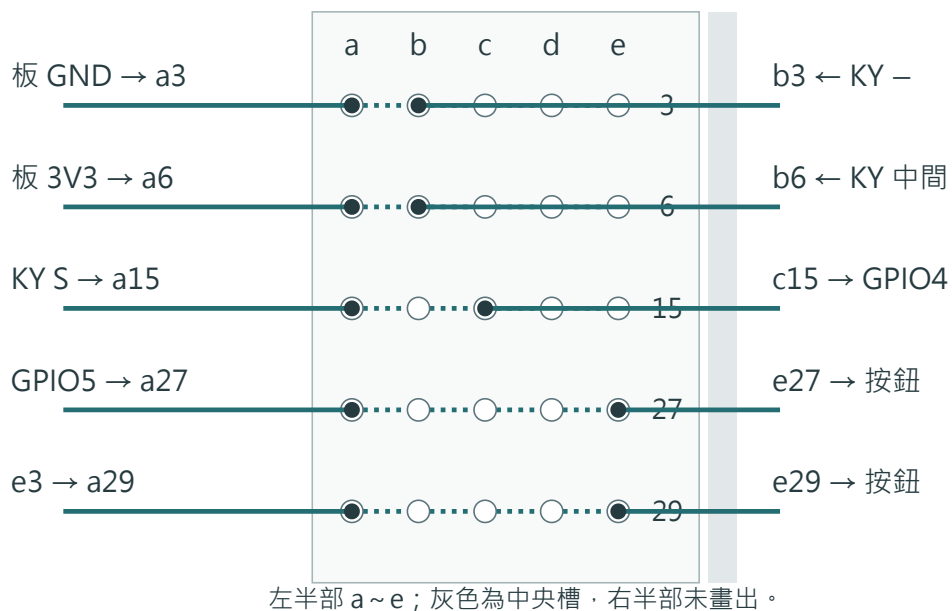
設定	圖示板型已確認後的接法
PIN_LIGHT = 4	KY 的 S → a15；c15 → GPIO4
PIN_BUTTON = 5	GPIO5 → a27；a29 → e3
PROFILE_CONFIRMED = true	只在板型、腳序與 3.3 V 訊號已確認後設定
電源保持分列	板 GND → a3，KY — → b3；板 3V3 → a6，KY 電源 → b6

按鈕沿用 Week 2：四腳插 **e27**、**f27**、**e29**、**f29**，跨中央溝槽。第 27 列為一組，第 29 列為另一組。USB 未插時，量 b27 對 b29：放開不通、按下才通；不符合就先調整方向，不通電。a27 與 e27 相通，a29 與 e29 相通，無須把杜邦母頭套在按鈕短腳。

原檔腳位是 -1、確認值 false。圖示 4 / 5 僅供已確認的 YD-ESP32-S3 板使用。GPIO5 設成 INPUT_PULLUP，不是把 3V3 電源直接接到按鈕。

按鈕與光敏各接一個 GPIO

按鈕與光敏共用 GND，但訊號線分開。



- 按鈕四腳插 e27、f27、e29、f29，跨過麵包板中央溝槽。
- a27 與 e27 相通；a29 與 e29 相通；a15 與 c15 相通。
- KY 中間供電腳接 b6；不是 GPIO5，也不是 5Vin。

接 USB 前核對：第 6 列是 3V3；第 3、29 列是 GND；第 15 列是光敏訊號；第 27 列是按鈕訊號。第 15、27 列不能互接。

打開、蓋上，各按一次

保持感測器位置不動，只有遮光條件改變。

接 COM USB，Monitor 選 115200。先放開按鈕至少 0.1 秒，再按住約半秒。

操作	預期結果
室內光，按一次	sample=1，一筆 raw
按住兩秒	不新增資料
放開，遮光，再按一次	sample=2，一筆新的 raw
按 RST	重新顯示 ready，記錄序號重新開始
開機時壓住按鈕	放開後重新按，才記錄第一筆

預期輸出格式；raw 與時間是示例，不是實測：

```
event=sample sample=1 raw=420 endpoint=0 uptime_ms=3040
event=sample sample=2 raw=1020 endpoint=0 uptime_ms=6040
```

sample 是記錄序號；uptime_ms 是開機後毫秒；endpoint=1 表示 raw 是 0 或 4095，不直接證明斷線。依第 22 頁將文字另存成檔案。

沒有 ready：查 Port、115200 與 Upload。出現 check_configuration：斷電核對板型、兩個不同腳位與 PROFILE_CONFIRMED。

button_light_capture.ino

第 1 ~ 26 行。

```
#include <Arduino.h>

// 未確認實物與斷電接線前，保持停用。
const bool PROFILE_CONFIRMED = false;
const int PIN_LIGHT = -1;
const int PIN_BUTTON = -1;
const uint32_t DEBOUNCE_MS = 40;

bool ready = false, armed = false;
int lastButton = HIGH;
uint32_t changedAt = 0, sampleNumber = 0;

// 讀值穩定 40 ms 才接受；放開後，下一次按下才再回傳 true。
bool pressed(uint32_t now) {
    const int level = digitalRead(PIN_BUTTON);
    if (level != lastButton) {
        lastButton = level;
        changedAt = now;
    }
    if (uint32_t(now - changedAt) < DEBOUNCE_MS) return false;
    if (level == HIGH) { armed = true; return false; }
    if (!armed) return false;
    armed = false;
    return true;
}
```

同一份 .ino 檔案；分頁不代表另開程式。

button_light_capture.ino

第 27 ~ 53 行。

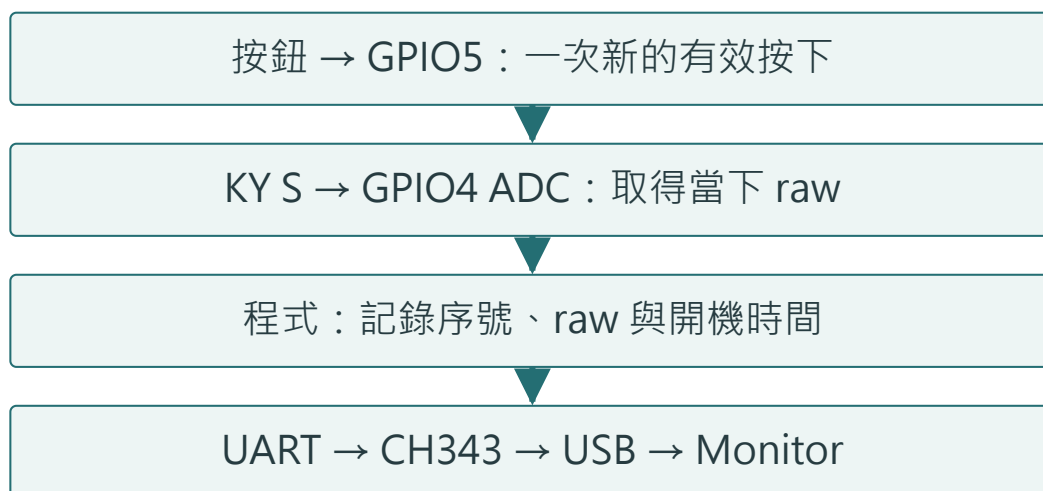
```
void setup() {
  Serial.begin(115200);
  if (!PROFILE_CONFIRMED || PIN_LIGHT < 0 ||
      PIN_BUTTON < 0 || PIN_LIGHT == PIN_BUTTON) {
    Serial.println("event=blocked reason=check_configuration");
    return;
  }
  pinMode(PIN_BUTTON, INPUT_PULLUP); // 放開 HIGH · 按下 LOW。
  analogReadResolution(12);           // ADC 原始值：0~4095。
  analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
  changedAt = millis();
  ready = true;
  Serial.println("event=ready release_button_first");
}

void loop() {
  if (!ready) return;
  const uint32_t now = millis();
  if (!pressed(now)) return; // 沒有新的有效按壓，就不取樣。

  const int raw = analogRead(PIN_LIGHT); // 按下時才讀一次光敏訊號。
  ++sampleNumber;
  Serial.printf("event=sample sample=%lu raw=%d "
               "endpoint=%d uptime_ms=%lu\n",
               (unsigned long)sampleNumber, raw,
               raw == 0 || raw == 4095, (unsigned long)now);
}
```

同一份 .ino 檔案；分頁不代表另開程式。

按鈕決定何時，感測器決定數字



digitalRead 讀 GPIO5 的 HIGH / LOW，判斷按鈕。**analogRead** 讀 GPIO4 的原始數字，取得光敏訊號。它們讀的是不同腳位、不同內容。

程式部分	對應剛完成的操作
<code>pressed(now)</code>	只在一次新的有效按下回傳 true
<code>analogRead(PIN_LIGHT)</code>	在那次按下時取一筆 raw
<code>++sampleNumber</code>	記錄序號加一
<code>Serial.printf</code>	把序號、raw 與時間送到 Monitor

Monitor 顯示的是已送出的歷史文字，不會因光線繼續改變而改寫上一行。要看新資料，放開再按一次。

兩條電流路徑，兩個訊號接點

按鈕按下時：傳統電流方向



光敏電路：供電與感測接點



按鈕按下：GPIO5 經按鈕連到 GND，電壓接近 0 V，讀到 LOW；晶片內的上拉電阻限制這條路徑的電流。

按鈕放開：按鈕這一段不通，GPIO5 由晶片內上拉電阻連著 3.3 V，電壓接近 3.3 V，讀到 HIGH。

光敏電路持續由 3V3 供電。S 是分壓的中間接點，GPIO4 讀取它相對 GND 的電壓。GPIO4 不替模組供電，也不要額外用線把 GPIO4 接到 GND。

為什麼按住不會一直印？

程式不只看 LOW，還記住是否已經放開。

按鈕狀態	程式記住的狀態	處理結果
放開且穩定	armed = true	準備接受下一次按下
按下且穩定，armed 為 true	改成 false	記錄一筆
繼續按住，armed 為 false	維持 false	不新增記錄

armed 是本程式的變數名稱，表示已準備接受新的按下。它不是另一個零件。

按鈕接觸的瞬間可能短暫跳動。程式記下讀值改變的時間 `changedAt`；同一讀值維持 40 ms 才接受，這種處理叫去抖動。40 ms 是本例選擇，不是所有按鈕的固定規格。

`millis()` 是開機後經過的毫秒；`now - changedAt` 是本次讀值已維持多久。loop 持續讀按鈕。

開機時 `armed = false`。即使當時按住按鈕，也須先放開、再按，才留下第一筆資料。

V、A、Ω，不要混在一起

本週用電表量 V 和 Ω；A 只在紙上計算。

問什麼	名稱與單位	例子
兩點電位差多少	電壓 V，伏特	3.3 V
電荷流動多少	電流 A，安培	0.01 A = 10 mA
對電流阻礙多少	電阻 Ω，歐姆	330 Ω；1 kΩ = 1000 Ω

單一電阻的紙上例子：330 Ω 兩端相差 3.3 V，則 $I = 3.3 \div 330 = 0.01 \text{ A}$ ，不是 0.1 A。若 $R = 1 \Omega$ ，理想計算才是 3.3 A；沒外接電阻不能自行當成 1 Ω。

A830L 常見 Ω 量程判讀示例

檔位	示意畫面	解讀
200 Ω	61.0	約 61.0 Ω
20k Ω	1.00	約 1.00 kΩ = 1000 Ω
200k Ω	10.0	約 10.0 kΩ = 10000 Ω
任一 Ω 檔	最左側 1，或其他表的 OL	可能超量程或未接通，不是一般的 1 Ω

實際顯示形式依表型而異，紀錄「61」卻不寫檔位，別人就無法確定單位。電阻檔不是把電壓讀值換成另一個單位。

有聲音就短路嗎？同一五孔組相通，本來就會叫；供電與 GND 原本不該直接相通，卻有低阻路徑，才是需要停止供電排查的情況。

上傳成功，不等於感測成功

三個檢查，回答三個不同問題。

看到什麼	代表什麼	還沒證明什麼
Verify 沒錯誤	電腦能編譯程式	實際接線、電壓與模組功能
Upload 完成	程式已寫入板子	光線分類必定正確
Monitor 與量測符合測試條件	這輪觀察有對應證據	所有器材、環境與時間都可靠

Flash 與 RAM 不要把百分比相加

```
Sketch uses 286973 bytes (21%) ... Maximum is 1310720
Global variables use 21936 bytes (6%) ... Maximum is 327680
```

這是編譯摘要格式示例，不是本次程式的編譯結果。

Flash 存程式，斷電仍保留。第一行是在看目前分割設定給程式的空間，不是整顆 16 MB 都拿來當分母。

RAM 是執行時暫存。第二行是編譯時計入的靜態配置，不是之後執行時永遠只用 6%，也不是板上 8 MB PSRAM 的用量。

PSRAM 是額外的 RAM。不是 Flash，也不能由上面兩行就判斷它用了多少。21% 和 6% 分母不同，不能加成整片板用了 27%。

幫一個遮光提醒器找錯

只在紙上作答，不製造短路，也不更動硬體。

同一組設備、同一位置，室內光十筆範圍 **400 ~ 440**；遮光十筆範圍 **1000 ~ 1040**。以下是為練習設計的資料。

1. 用兩段相近邊界的中點算分界。raw 等於分界時，依本課規則判哪一類？
2. 新 raw=430、800、4095，各會印什麼標籤？哪筆落在兩段基準之間？哪筆先不判？
3. 同學說「raw=800 就是 0.8 V，而且一定遮光」。這句話哪兩個地方不能成立？
4. 供電 3.3 V，上方 1 k Ω 、下方 10 k Ω 。中間相對 GND 多少 V？電表選什麼檔、紅黑筆各碰哪裡？
5. 同學將紅筆插 10A，打算照量電壓的方法跨接 3V3 與 GND。通電前你要阻止哪件事？

題目	我的答案或計算
1	_____
2	_____
3	_____
4	_____
5	_____

每題指出使用的數字、接點或量測條件。

按一次，自動取三筆

用按鈕記錄紙盒打開與蓋上的光線，每次保留三筆資料。

每次辨識到有效按下後，取第一筆；後續每隔至少 200 ms 各讀一次感測器，共三筆。一批就是這次按下啟動的三筆資料，batch 是批次編號，index 是這批的第幾筆。

以 button_light_capture 為起點，另存為 **three_light_samples**。接線、光敏設定與按壓辨識保持不變，增加定時取樣及完成後停止的功能。三筆都要重新讀感測器，不能複製同一個 raw。

作品規則	要完成的行為
按下	第一筆不額外等 200 ms
持續按住	三筆完成後不再取樣
取樣期間放開	仍完成這批三筆
三筆完成後放開、再按	新增一批，index 從 1 開始
一批尚未完成時又按	印 event=ignored reason=batch_busy，不排隊
按 RST	清除進度；放開再按，從 batch=1 開始

感測器位置保持固定。室內光做一批，再遮光做一批，保存兩批原始紀錄。比較 raw 的變化，但不要把 raw 當成已校準的照度。

兩次按壓，六筆資料

接 USB，Monitor 選 115200；Reset 後先放開按鈕至少 0.1 秒。每次按住約半秒，放開至少半秒。

操作順序	預期新增資料
1. 室內光下按住	batch=1，index=1
2. 保持按住，至少再等 200 ms	batch=1，index=2
3. 保持按住，至少再等 200 ms	batch=1，index=3
4. 繼續按到兩秒	不新增資料
5. 放開，遮光，再按住	batch=2，index=1
6. 至少再等 200 ms	batch=2，index=2
7. 至少再等 200 ms	batch=2，index=3

raw 與 uptime_ms 以本次量測為準，不要求複製示例數字。相鄰取樣間隔至少 200 ms，不保證剛好相等。提早放開仍應完成三筆。

預期格式示例；第二次完成後，應另有 batch=2 的三行：

```
event=sample batch=1 index=1 raw=420 endpoint=0 uptime_ms=3040
event=sample batch=1 index=2 raw=422 endpoint=0 uptime_ms=3240
event=sample batch=1 index=3 raw=419 endpoint=0 uptime_ms=3440
```

練習解答：[week3Ans.pdf](#)。

保存量測與程式紀錄

照片、數值、程式與解釋放在同一份紀錄。

保存什麼	對應本冊段落
板型、GPIO、IDE 版本、組別、電表檔位	器材及操作設定
電源 / GPIO 接線照、3V3 一筆、LOW / HIGH 各三筆、Reset 文字	電源及 GPIO 量測
KY 絲印、三組斷電阻值、接線照、S 電壓三筆	光敏模組辨認及電壓
同輪室內 / 遮光各十筆 raw、條件與電壓	原始資料及範圍
四個範圍值、分界方向、各五筆新驗證	分類規則及限制
1 k Ω / 10 k Ω 實測、A / B 預測與實測、兩張接線照	固定電阻分壓
實際使用的程式、討論及練習答案、尚未完成項目	三支量測 / 分類程式，加上按鈕單筆取樣

整理順序：條件與接法 → 原始資料 → 比較 → 原因與限制。已有資料直接引用，不為換標題再拍照或重收。未完成寫「未量測 / 未確認」，不填理論值。

最後一起斷電收納

1. 移開表筆、關閉 Monitor、拔 USB 與其他電源，確認 PWR 熄滅。
2. 電表 OFF。若停在分壓範例，先拆 b6 → a20，再拆電阻；若已做按鈕取樣，分壓線已拆除，只拆目前的 KY 與按鈕線。
3. 從板子拆下 3V3、GND 線。電阻按阻值收好，模組和線材分開放。

下週再把感測結果接上其他輸出與狀態判斷；本週不把一次分類成功當成可長期自動控制的系統。

操作依據，和還沒做的測試

ADC / GPIO 文件重新核對：2026-09-22；其他來源沿用既有查核。

- [Espressif Arduino-ESP32 ADC](#)：原始值、解析度、衰減與量測範圍。
- [Espressif GPIO](#)：輸入、輸出及 digitalWrite。
- [Espressif ESP32-S3 datasheet](#)：GPIO 與晶片電氣規格。
- [Fluke 電阻量測](#)：斷電與隔離被測物。
- [Fluke 通斷測試](#)：低阻路徑與蜂鳴判讀。
- [Analog Devices：Voltage Divider](#)：分壓電路。
- [Arduino Memory Guide](#)：儲存與執行時記憶體。

板卡、KY 模組、電表與電阻照片來自本課既有實物；板型與焊盤狀態依課堂硬體紀錄，不把賣場名稱當作實物規格。示意圖是依接線關係重新繪製，並非實物比例圖。

本稿整理操作順序與版面；**沒有新增上電、接線、電表讀值或感測器實機測試**。示例數字已標明用途，應填入自己觀察的地方仍保留空白。

教師課前須確認使用板型、GPIO5 輸出與 GPIO4 ADC 的適用接法。既有 BOARD-T01 的觀察不代表全班每片板子都已通過；本稿不解除公開程式中 -1 的保護設定。

四支基本程式全文列在各自操作段落。實際上傳開完整 .ino 檔，不只複製中間一頁。兩份練習的解答另列於 week3Ans.pdf。